

# OOP

Object oriented programing

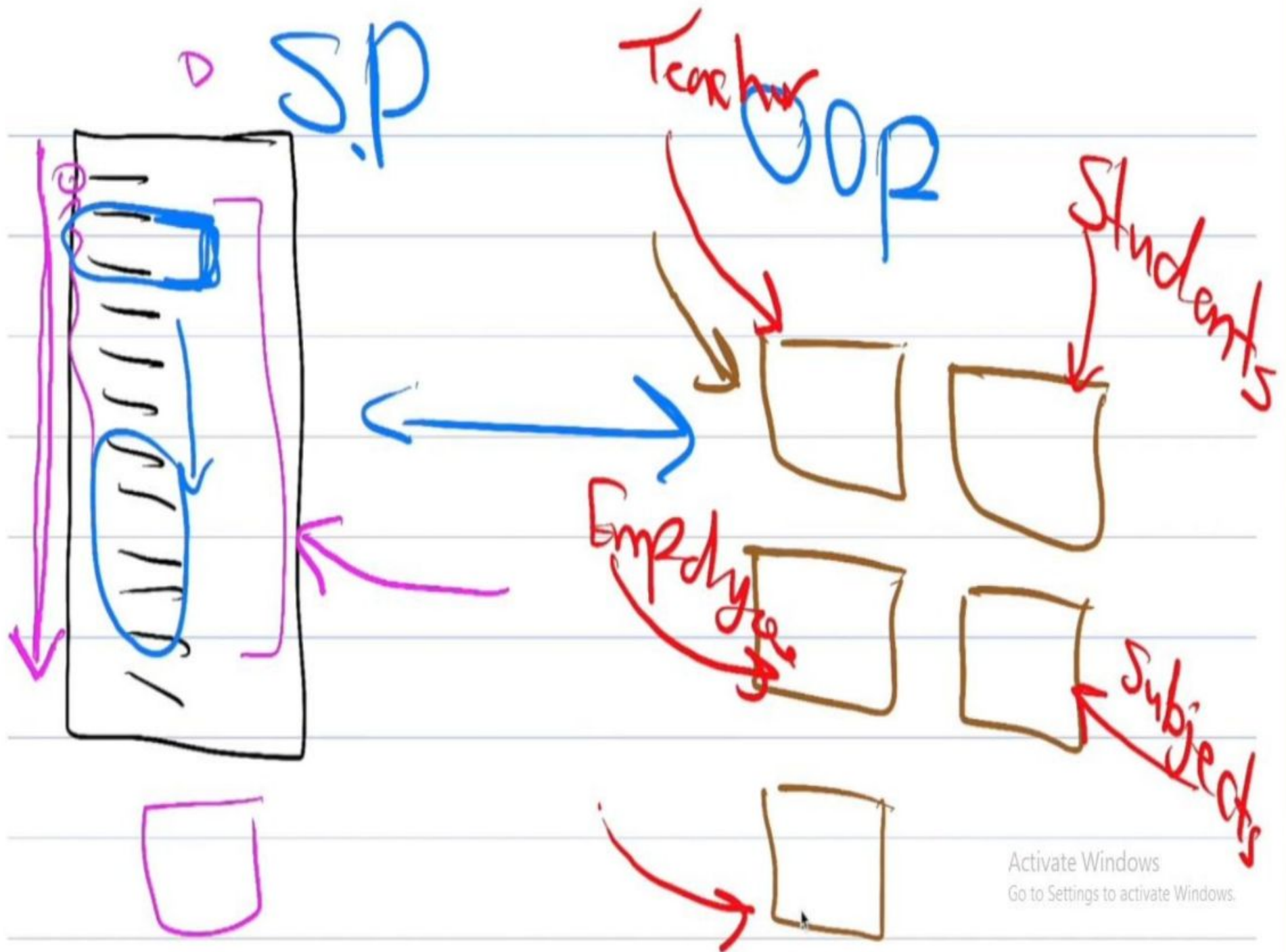
# Object Oriented Programming (OOP)

Object-oriented programming – As the name suggests uses objects in programming. Object-oriented programming aims to implement real-world entities like inheritance, hiding, polymorphism, etc in programming.



hiding





## Structured Programming

1. The program is represented as a logical structure.
2. The flow of execution of the programming is dependent on the structure of the program.
3. Code is given more importance.
4. Can handle up to moderately complex programs.
5. Less code reusability.
6. Less data security.
7. Abstraction is less.
8. Flexibility is less.

## Object Oriented Programming

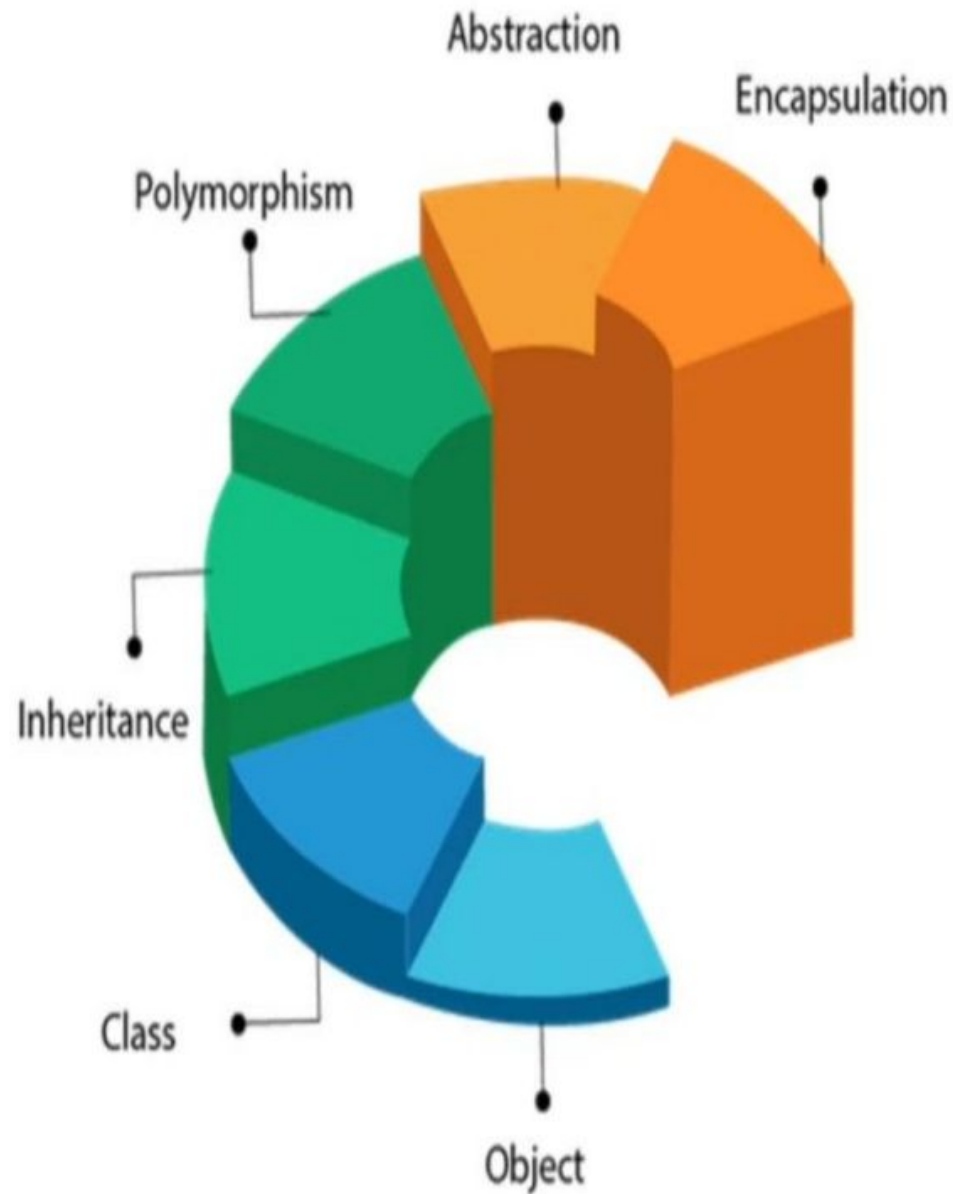
- The program is written as a collection of objects which communicate with each other.
- The basic entity is object. Each computation is performed using objects only.
- Data is given more importance.
- Can handle very complex programs.
- More code reusability.
- More data security.
- Abstraction is more.
- Flexibility is more.

Activate Windows  
Go to Settings to activate Windows.





# OOPs (Object-Oriented Programming System)



Activate Windows  
Go to Settings to activate Windows.



Anything  
In  
The  
World  
Is  
An  
**OBJECT.**



Activate Windows  
Go to Settings to activate Windows.



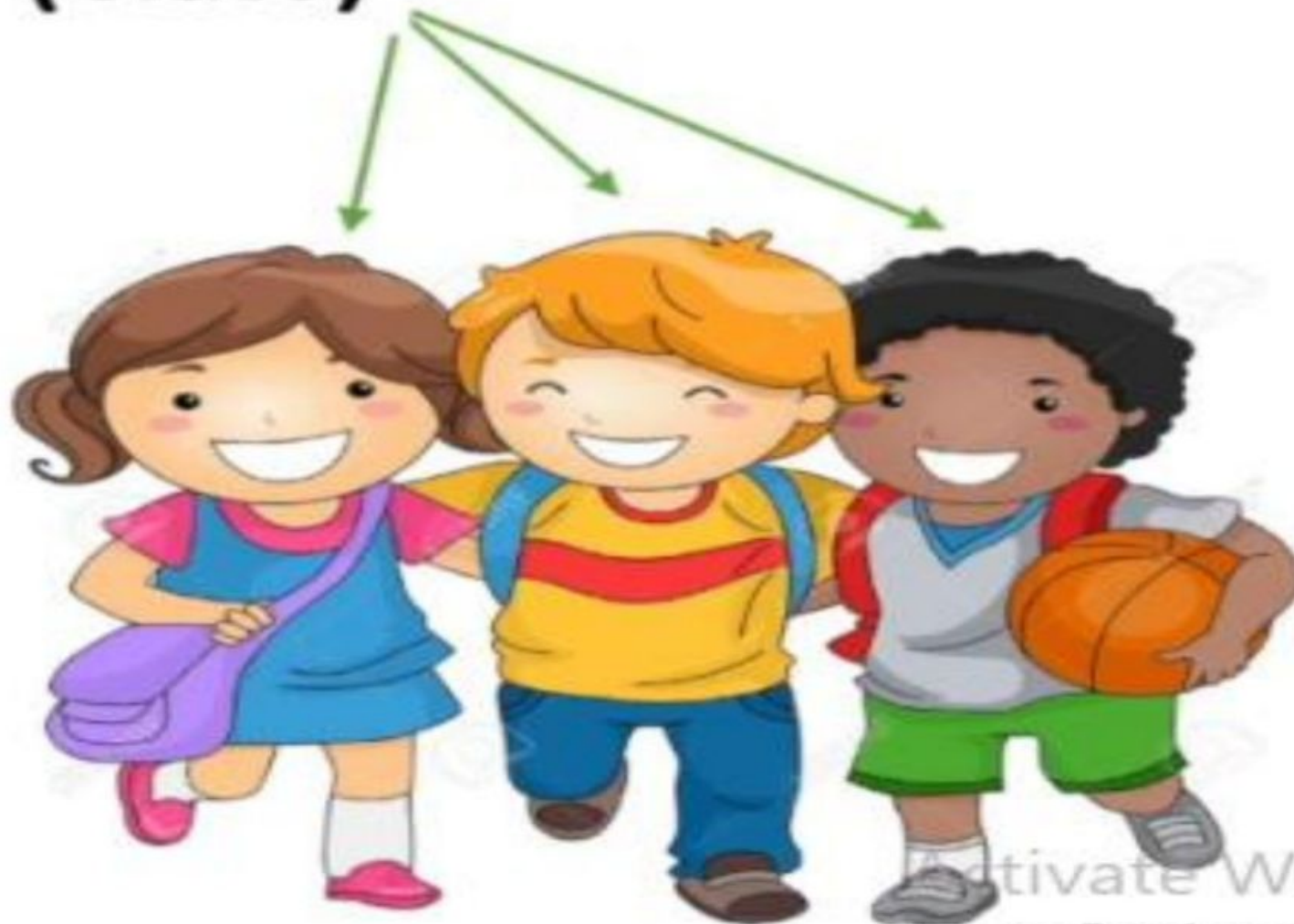
# Class VS Object

- **Object:** It is a basic unit of Object-Oriented Programming and represents the real-life entities. An object is a thing.
- **Class:** A category of *objects*. The class defines all the common properties of the different objects that belong to it.
- An **Object** is an instance of a Class.

Activate Windows  
Go to Settings to activate Windows.



# Student (Class)



Activate Windows  
Go to Settings to activate Windows.



| Class                                                                                                                                                                                                                                                                                                                                                                                                           | Object                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p data-bbox="165 196 1052 316">1) Class is a collection of similar objects</p> <p data-bbox="165 387 909 507">2) Class is conceptual (is a template)</p> <p data-bbox="165 579 1021 699">3) No memory is allocated for a class.</p> <p data-bbox="165 770 976 890">4) Class can exist without any objects</p> <p data-bbox="165 962 902 1129">5) Class does not have any values associated with the fields</p> | <p data-bbox="1115 196 1881 316">1) Object is an instance of a class</p> <p data-bbox="1115 387 1550 443">2) Object is real</p> <p data-bbox="1115 579 1832 699">3) Each object has its own memory</p> <p data-bbox="1115 770 1899 890">4) Objects can't exist without a class</p> <p data-bbox="1115 962 1848 1129">5) Every object has its own values associated with the fields</p> |

Car class



Company = ?

Type = ?

Color = ?

Car  
Objects



Company = BMW

Type = 5 Series

Color = Green



Company = Dodge

Type = Challenger

Color = Blue



Company = KIA

Type = OPTIMA

Color = Red

# What is a Template?



Activate Windows



**Cake Pan (Class)**



Activate Windows  
Go to Settings to activate Windows.

# Oop

## Lecture 2



**Car (Class)**



**Company = ?**

**Type = ?**

**Model = ?**

**Color = ?**

...

**obj1**



**Company = Mercedes-Benz**

**Type = CLA-Class**

**Model = 2018**

**Color = White**

**obj2**



**Company = Land Rover**

**Type = Range Rover**

**Model = 2017**

**Color = Red**

**obj3**



**Company = BMW**

**Type = X5**

**Model = 2017**

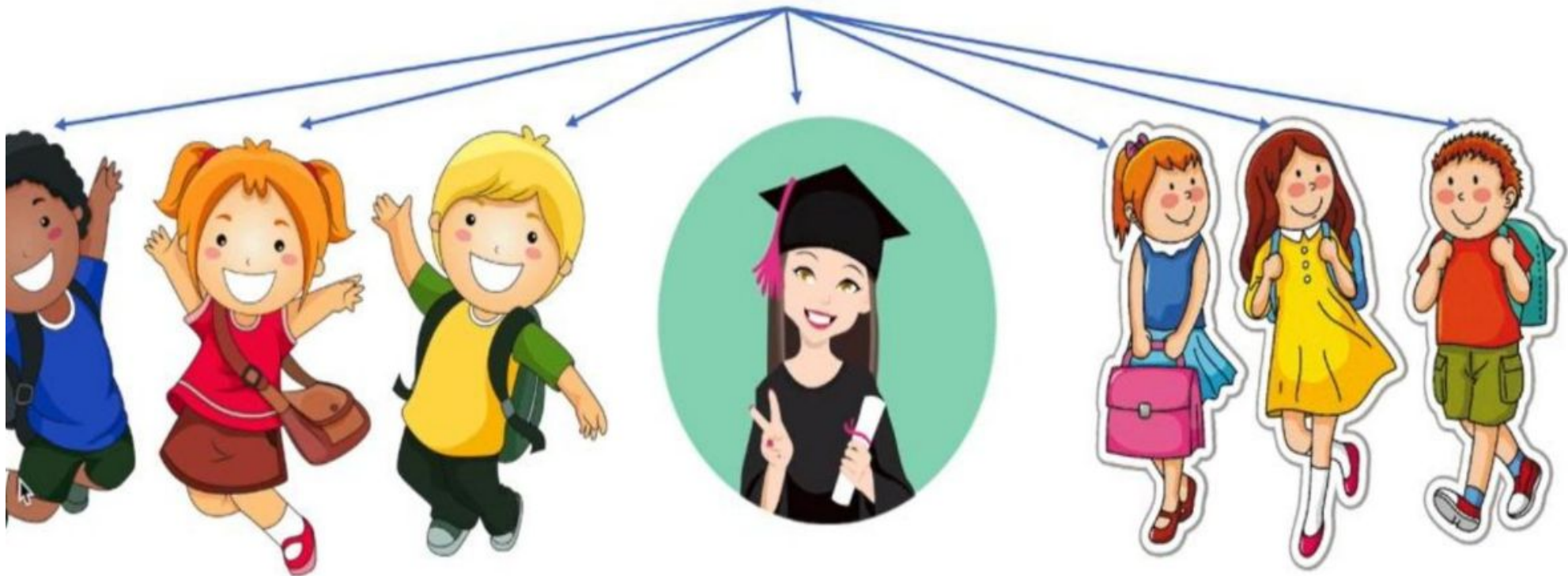
**Color = White**

Activate Windows  
Go to Settings to activate Windows.

Student (Class)



ID = ?  
Name = ?  
...



100  
Ali

101  
Sara

102  
Ahmed

400  
Ola

200  
Noor

201  
Ban

202  
Samir

Activate Windows  
Go to Settings to activate Windows.

Book (Class)



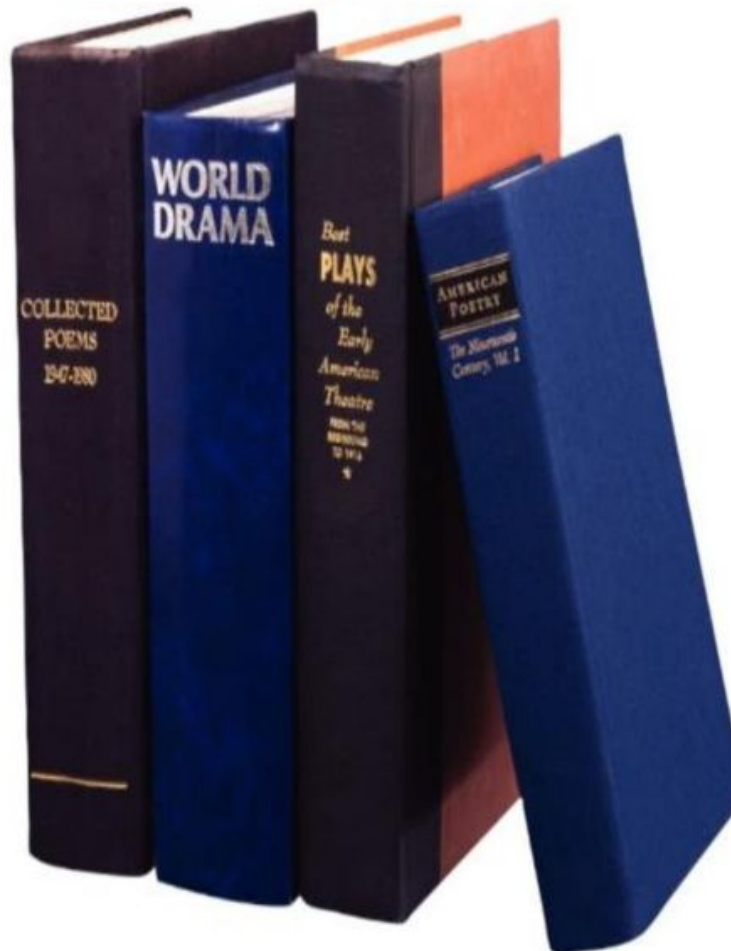
ID = ?  
Title = ?  
Author = ?

...



ID = XXX  
Title = XXX  
Author = XXX

...



Activate Windows  
Go to Settings to activate Windows.

**Phone (Class)**



**Company = ?**

**Name = ?**

**Screen = ?**

**Price = ?**

...

Activate Windows

Go to Settings to activate Windows.



Phone (Class)



Company = ?

Name = ?

Screen = ?

Price = ?

...



Activate Windows  
Go to Settings to activate Windows.

Pen (Class)



Type = ?

Color = ?

Price = ?

...



Active windows.  
Go to the Start menu.

(Class)

**Animal**

Activate Windows  
Go to Settings to activate Windows.



(Class)

**Animal**

Objects ...

obj1



obj2



obj5



obj4



obj3





**Dog**



**Dog** is class

These are *objects*  
Of **Dog** class

1. Dog10bject
2. Dod20bject
3. Dog30bject

Activate Windows  
Go to Settings to activate Windows.

**Dog**



**Dog** is class

These are **Fields**  
Of **Dog** class  
(Attributes)

1. Breed
2. Size
3. Colour
4. Age

Activate Windows

Go to Settings to activate Windows.



**Dog**



**Dog** is class

These are  
*Methods* Of **Dog**  
class  
(Operations)

1. Eat()
2. Run()
3. Sleep()

Activate Windows  
Go to Settings to activate Windows.



**Dog**



**Dog** is class

These are  
**Methods** Of **Dog**  
class  
(Operations)

1. Eat()
2. Run()
3. Sleep()

Activate Windows  
Go to Settings to activate Windows.





Breed: Bulldog  
Size: large  
Colour: light gray  
Age: 5 years

Dog1Object



Breed: Beagle  
Size: large  
Colour: orange  
Age: 6 years

Dog2Object



Breed: German Shepherd  
Size: large  
Colour: white & orange  
Age: 6 years

Dog3Object

Dog

### Fields

Breed  
Size  
Colour  
Age

### Methods

Eat()  
Run()  
Sleep()  
Name()

Activate Windows  
Go to Settings to activate Windows.

# Class Members

```
graph TD; CM([Class Members]) --- DM[Data Members]; CM --- MF[Member Functions];
```

## Data Members

```
int x;  
char name[25];  
float average;  
string address;
```

...

## Member Functions

```
A() // Constructor  
~A() // Destructor  
set_name()  
get_name()  
print()
```

...



# Class Diagram

- In software engineering, a class diagram in the Unified Modeling Language (UML): is a type of static structure diagrams that describes the structure of a system by showing the system's **classes**, their **attributes**, **operations** (or methods), and the relationships among objects.

# Class Notation

- **Class Name**

- The name of the class appears in the first partition.

- **Class Attributes**

- Attributes are shown in the second partition.
- The attribute type is shown after the colon (:).
- Attributes map onto member variables (**data members**) in code.

- **Class Operations (Functions)**

- Operations are shown in the third partition. They are services the class provides.
- The return type of a function is shown after the colon (:) at the end of the function signature (header).
- The types of function's parameters are shown inside ().
- Operations map onto class functions (**member functions**) in code.

Activate Windows  
Go to Settings to activate Windows.



# The Structure of Class in Class Diagram



Activate Windows  
Go to Settings to activate Windows.

# Encapsulation

Data encapsulation refers to the process of *binding* together **data** and **functions** or methods operating on this data into a single unit so that it is protected from outside interference and misuse.



Activate Windows  
Go to Settings to activate Windows.



**Data members**

**Member functions**



# To Achieve Encapsulation:



Activate Windows  
Go to Settings to activate Windows.





Private = inside <sup>the class</sup> only.

**Public** = inside and outside  
the class

Member  
function

## To Achieve Encapsulation:



← private

← public

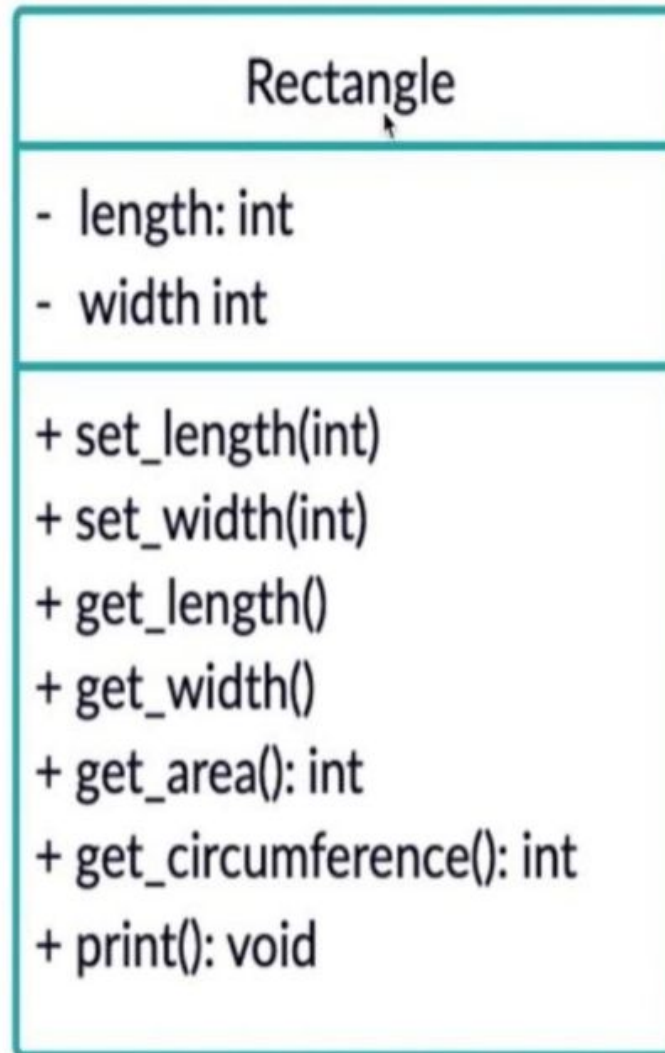
Activate Windows  
Go to Settings to activate Windows.



# Example bank

- Money
- Set=input
- Get =out
- -= private
- +=public

## Example 1:



Activate Windows  
Go to Settings to activate Windows.



## Example 1:

| Rectangle                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- length: int</li><li>- width int</li></ul>                                                                                                                                            |
| <ul style="list-style-type: none"><li>+ set_length(int)</li><li>+ set_width(int)</li><li>+ get_length()</li><li>+ get_width()</li><li>+ get_area(): int</li><li>+ get_circumference(): int</li><li>+ print(): void</li></ul> |

Activate Windows  
Go to Settings to activate Windows.

```

3  using namespace std;
4
5  class Rectangle
6  {
7  private:
8      int length;
9      int width;
10
11 public:
12     void set_length(int l)
13     {
14         length = l;
15     }
16     void set_width(int w)
17     {
18         width = w;
19     }
20     int get_length()
21     {
22         return length;
23     }
24     int get_width()
25     {
26         return width;
27     }
28     int get_area()
29     {
30         return length * width;
31     }
32
33     int get_cir()
34     {
35         return 2 * (length + width);
36     }
37     void print()
38     {
39         cout<<"Length = "<<length<<endl;
40         cout<<"Width = "<<width<<endl;
41     };
42
43 int main()
44 {
45     Rectangle r1;
46     r1.set_width(20);
47     r1.set_width(10);
48
49     Rectangle r2;
50     r2.set_length(100);
51     r2.set_width(130);
52 }
53
54

```

Rectangle

2  
w  
h  
?

$r_1$

$r_2$

$$L = 20$$

$$w = 10$$

$$L = 100$$

$$w = 130$$

## Student

- id: int
- name: char[]
- stage: int
- address: string

- + Student ()
- + Student (int i, string)
- + Student (int,char[],int,string)
- + set\_ID (int)
- + get\_ID (): int
- + set\_stage (int)
- + get\_stage (): int
- + set\_name (char[])
- + get\_name (): char[]
- + set\_address (string)
- + get\_address (): string
- + print (): void

## Book

- id: int
- title: char[]
- author: char[]
- publisher: string
- number\_of\_pages: int

- + Book(int, char[], char[], string, int)
- + print(): void



# To create object:

- `Class_name obj_name;`
- `Class_name obj_name();`